

Running Ollama Locally with Podman on Fedora Linux

Posted by [Yazan Monshed](#) on [August 5, 2026](#)

[4 Comments](#)



[Photo by Mark Hofman on Unsplash](#)



Fedora Linux 44

Fedora Linux 44 is available now. Read the [release announcement](#) for all the details.

**SUBSCRIBE TO FEDORA
MAGAZINE VIA EMAIL**

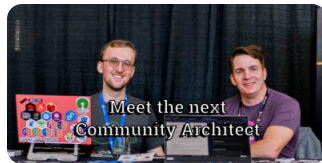
Subscribe

Join 10.8K other subscribers

RECENT POSTS →



Developing with Fedora, first flock, and not the last!



Announcing the next Fedora Community Architect



Flock 2026 Afterburn: a retrospective

Running Large Language Models (LLMs) locally has become increasingly popular for development, privacy, and offline testing. Ollama makes this incredibly straightforward, allowing you to run models like Llama 3 or Mistral directly on your machine.

By leveraging Podman on Fedora Linux, you can isolate Ollama inside a container. This approach keeps your host system clean while making it effortless to spin up, manage, and tear down your AI development environment.

What is Ollama?

Ollama is an open-source framework designed for running, creating, and sharing large language models. It packages model weights, configuration, and data into a unified management system. Running it inside a container means you don't have to deal with complex local dependencies, Python environments, or complex GPU driver configurations on your base OS.

CONTRIBUTE TO THE MAGAZINE

Fedora Magazine is looking for contributors!

[Propose a new article](#)

[Become a writer](#)

[Become an editor](#)

Verify or Install Podman

Podman is available by default in Fedora Workstation. It can be easily install, if missing, using DNF:

```
$ sudo dnf install podman -y
```

For Fedora Linux Silverblue users, Podman is natively available in the immutable base system and no extra steps are necessary.

To verify your installation and ensure everything is running smoothly, execute a quick check:

```
$ podman --version
```

Step 1: Create a Persistent Volume for Your Models

LLM weights can be huge—often ranging from 4 GB to over 40 GB, depending on the model size. To avoid downloading these models every time you restart your container, create a persistent Podman volume to store them safely on your host disk:

```
$ podman volume create ollama_storage
```

Step 2: Run the Ollama Container

Next, spin up the Ollama container. The following command pulls the official image, attaches the volume we just created, and maps the communication port (

11434
) to your host machine.

```
$ podman run -d \  
  -v ollama_storage:/root/.ollama \  
  -p 11434:11434 \  
  --name ollama \  
  ollama/ollama
```

Note on Hardware Acceleration

The command above runs Ollama using your CPU. If you are on Fedora Workstation or Silverblue and want to pass through an Nvidia GPU for fast hardware acceleration, make sure you have the Nvidia Container Toolkit installed and append the GPU flag:

```
--device nvidia.com/gpu=all
```

Step 3: Download and Run an AI Model

With the container running in the background, you can interact with it using Podman's execution command. Let's pull and run **Llama 3**, a highly capable, lightweight model perfect for local development:

```
$ podman exec -it ollama ollama run llama3
```

The first time you execute this, Podman will download the model weights into your

ollama_storage volume. Once the download completes, you will be dropped directly into an interactive terminal prompt:

```
>>> Send a message (/? for help)
>>> Tell me a fun fact about Fedora Linux.
Fedora Linux is named after the iconic felt
hat worn by the Red Hat shadowman logo! It
started as a community project to provide
extra packages for Red Hat Linux.

>>>
To exit the interactive prompt, simply type
/exit.
```

Step 4: Interact with the Local API

Because we mapped port 11434 to our host system, you can also interact with your local Ollama instance via its built-in REST API. Open a standard terminal window and send a `curl` request:

```
curl http://localhost:11434/api/generate -d '{
  "model": "llama3",
  "prompt": "Why use containers?",
  "stream": false
}'
```

This returns a structured JSON payload containing your answer, allowing you to easily hook your local model up to web apps, scripts, or IDE extensions.

Checking Container Status

To monitor your running local AI instance, use the classic Podman management commands, perhaps starting with:

```
$ podman ps
```

You can also inspect the logs to make sure the API server is listening properly:

```
$ podman logs ollama
```

When you are done with your development session and want to free up system memory, stop the container:

```
$ podman stop ollama
```

If you ever need to completely remove the container environment, use:

```
$ podman rm ollama
```

Note: Your downloaded models are completely safe inside the `ollama_storage` volume and will instantly reattach the next time you spin up the container.

Conclusion

Using Podman to manage Ollama on Fedora Linux or Fedora Silverblue offers a clean, containerized way to build and test applications with LLMs completely offline. It bypasses host environment pollution, isolates large model storage cleanly into a named volume, and treats your AI stack exactly like any other microservice.

FEDORA PROJECT COMMUNITY

USING SOFTWARE



OLLAMA



PODMAN





Yazan Monshed

MLOps Engineer | Go •
Kubernetes • Cloud Native |
Building Scalable ML Platforms

REPLIES FROM DISCUSSION.FEDORAPROJECT.ORG

Martin Sehnoutka
AUGUST 5, 2026

Can I also utilize NPU instead of CPU/GPU?

SD
AUGUST 5, 2026

I mean if you want to use some kind of LLM I think it's far easier to just use llamafiles. - [Pre-built llamafiles](#) | [llamafile](#) | [Mozilla.ai Docs](#)

All you do is download a model and then run it like an executable. Gives you access on the CLI and/or via a webpage (runs on port 8080 or something by default IIRC.)

Sample systemd file you can proxy to with nginx, etc.

/etc/systemd/system/llamafire.service

[Unit]

Description=Llamafire Service

[Service]

Type=simple

User=llama

Group=llama

ExecStart=/bin/sh /opt/llama/llamafire/llava-v1.6-mistral-7b-

Q4_K_M.llamafire --port 9998 --host 0.0.0.0 --server

Restart=on-failure

StandardOutput=journal

StandardError=journal

[Install]

WantedBy=multi-user.target

fatka

AUGUST 5, 2026

You need dedicate support in the runtime for NPU based inference. Unfortunately people gravitate towards ollama because of its low starting threshold, but it does not include the latest developments in various runtimes, not to mention they have a poor history of crediting other FOSS projects it builds on.

Anyway, to answer your question, your best bet is to try the latest [llama.cpp release](#) for your hardware, or if you have AMD hardware you could try [lemonade](#). For both, you would need to do some digging to

ensure you have enabled it with the right CLI flag or config option. There's also [FastFlowLM](#), but I have not tried it yet.

[SeeM](#)

AUGUST 5, 2026

Hi. Thanks for podman configuration for Ollama. I'm using it on a bare metal right now, since it should occupy the box almost completely.

I found that Ollama is often not a problem, but clients are. Keeping models from interacting too much with my files is constantly pushing me back from AI. So I did build my own images with Aider.

Dockerfile

```
from quay.io/centos/centos@sha256:f964a3791678e692fcb4beacc50

USER 0
RUN dnf --assumeyes install python3 python3-pip python3-devel

RUN useradd -u 1001 aideruser
RUN usermod -L aideruser
RUN mkdir -p /opt/aider_ai
RUN chown -R 1001:0 /opt/aider_ai

USER 1001
WORKDIR /opt/aider_ai

RUN pip install --upgrade "pip==26.1.2"
ADD requirements.txt /tmp/
RUN pip install --requirement /tmp/requirements.txt
```

```
RUN git config --global user.name "AIDER AI"
RUN git config --global user.email "aideruser@localhost"

ENV PATH="${PATH}:/home/aideruser/.local/bin"
ENV AIDER_GIT=true
ENV AIDER_DARK_MODE=true
ENV OLLAMA_API_BASE=http://ollama.host.local:11434/
ENV AIDER_ANALYTICS_DISABLE=true
ENV AIDER_CHECK_UPDATE=false
ENV AIDER_SHOW_RELEASE_NOTES=false
ENV AIDER_YES_ALWAYS=true
ENV OLLAMA_MODEL=qwen3.5:9b

CMD aider --model=ollama_chat/${OLLAMA_MODEL}
```

Change ollama.host.local to Your dedicated (or You can dream of it) Ollama host.

requirements.txt

```
aider-chat==0.86.2
aiohappyeyeballs==2.6.1
aiohttp==3.13.3
aiosignal==1.4.0
annotated-doc==0.0.4
annotated-types==0.7.0
anyio==4.12.1
asgiref==3.11.1
attrs==25.4.0
backoff==2.2.1
```

```
beautifulsoup4==4.14.3
certifi==2026.1.4
cffi==2.0.0
charset-normalizer==3.4.4
click==8.3.1
ConfigArgParse==1.7.1
diff-match-patch==20241021
diskcache==5.6.3
distro==1.9.0
fastapi==0.128.8
fastuuid==0.14.0
filelock==3.20.3
flake8==7.3.0
frozenlist==1.8.0
fsspec==2026.2.0
gitdb==4.0.12
GitPython==3.1.46
grep-ast==0.9.0
h11==0.16.0
hf-xet==1.2.0
httpcore==1.0.9
httpx==0.28.1
huggingface_hub==1.4.1
idna==3.11
importlib_metadata==7.2.1
importlib_resources==6.5.2
Jinja2==3.1.6
jiter==0.13.0
json5==0.13.0
jsonschema==4.26.0
jsonschema-specifications==2025.9.1
litellm==1.81.10
```

```
markdown-it-py==4.0.0
MarkupSafe==3.0.3
mccabe==0.7.0
mdurl==0.1.2
mixpanel==5.0.0
mslex==1.3.0
multidict==6.7.1
networkx==3.4.2
numpy==1.26.4
openai==2.20.0
orjson==3.11.7
oslex==0.1.3
packaging==26.0
pathspec==1.0.4
pexpect==4.9.0
pillow==12.1.1
posthog==7.8.6
prompt_toolkit==3.0.52
propcache==0.4.1
psutil==7.2.2
ptyprocess==0.7.0
pycodestyle==2.14.0
pycparser==3.0
pydantic==2.12.5
pydantic_core==2.41.5
pydub==0.25.1
pyflakes==3.4.0
Pygments==2.19.2
py pandoc==1.16.2
pyperclip==1.11.0
python-dateutil==2.9.0.post0
python-dotenv==1.2.1
```

```
PyYAML==6.0.3
referencing==0.37.0
regex==2026.1.15
requests==2.32.5
rich==14.3.2
rpds-py==0.30.0
scipy==1.15.3
shellingham==1.5.4
shrtab==1.8.0
six==1.17.0
smmap==5.0.2
sniffio==1.3.1
socksio==1.0.0
sounddevice==0.5.5
soundfile==0.13.1
soupsieve==2.8.3
starlette==0.52.1
tiktoken==0.12.0
tokenizers==0.22.2
tqdm==4.67.3
tree-sitter==0.25.2
tree-sitter-c-sharp==0.23.1
tree-sitter-embedded-template==0.25.0
tree-sitter-language-pack==0.13.0
tree-sitter-yaml==0.7.2
typer==0.23.0
typer-slim==0.23.0
typing-inspection==0.4.2
typing_extensions==4.15.0
urllib3==2.6.3
watchfiles==1.1.1
wcwidth==0.6.0
```

```
yarl==1.22.0  
zipp==3.23.0
```

Build and run:

```
podman build -t aider .  
  
mkdir desk  
podman unshare chown 1001:0 desk  
podman unshare chmod 770 desk  
  
podman run --rm -it --env OLLAMA_MODEL='qwen3.5:9b' --name=aider
```

It has a few glitches, mainly I don't even use newer pip I pull in Dockerfile, but it can be done. Have fun!

CONTINUE THE DISCUSSION AT
[DISCUSSION.FEDORAPROJECT.ORG](https://discussion.fedoraproject.org)

The opinions expressed on this website are those of each author, not of the author's employer or of Red Hat. Fedora Magazine aspires to publish all content under a Creative Commons license but may not be able to do so in all cases.

You are responsible for ensuring that you have the necessary permission to reuse any work on this site. The Fedora logo is a trademark of Red Hat, Inc. [Terms and Conditions](#)